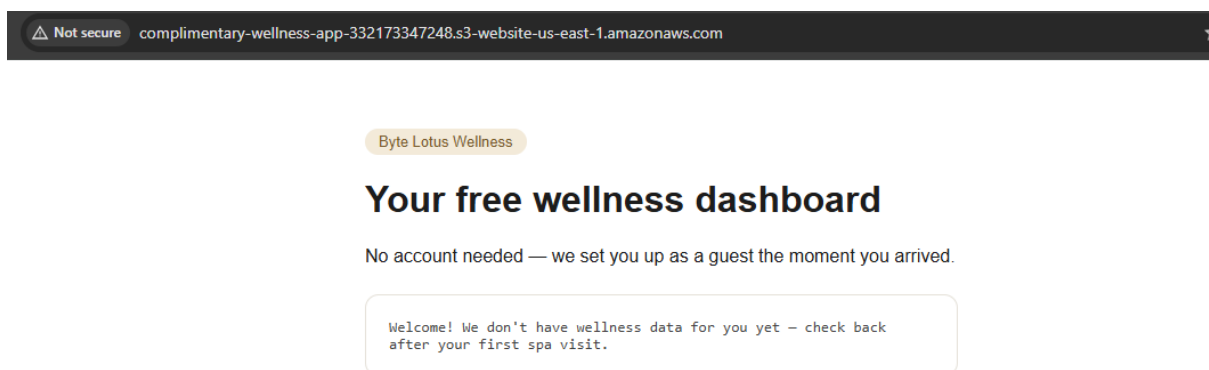


Complimentary THM Write-Up

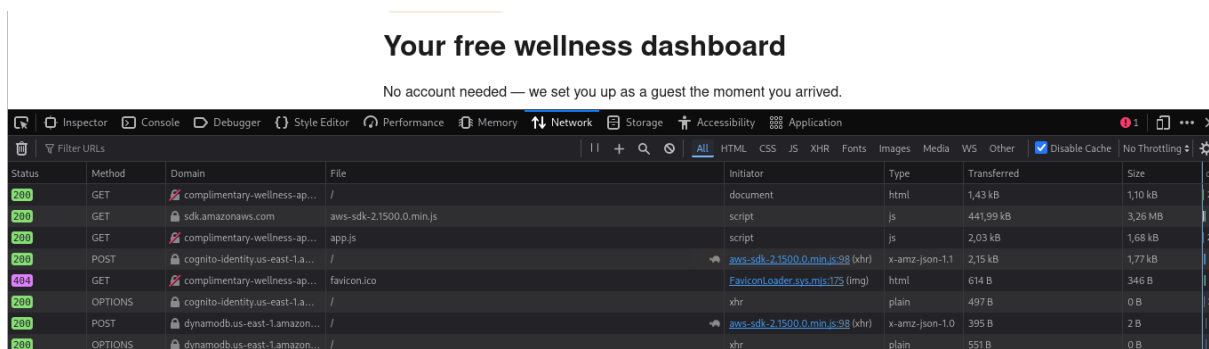
In this room, we are already given a lot of hints that we are dealing with a lot of AWS. Kali Linux has a tool for AWS (just type `aws --version` to check if it's there and the version; if it's not there, you will be prompted if you want it installed or not.)

As with every target, the first step is to look around, but this time we don't have an IP address, so `nmap` is a very unlikely tool to consider. A great way to start is to navigate to the given webpage: <http://complimentary-wellness-app-332173347248.s3-website-us-east-1.amazonaws.com/>

There we see nothing interesting: no parameters, nothing to click on, no login pages or anything.

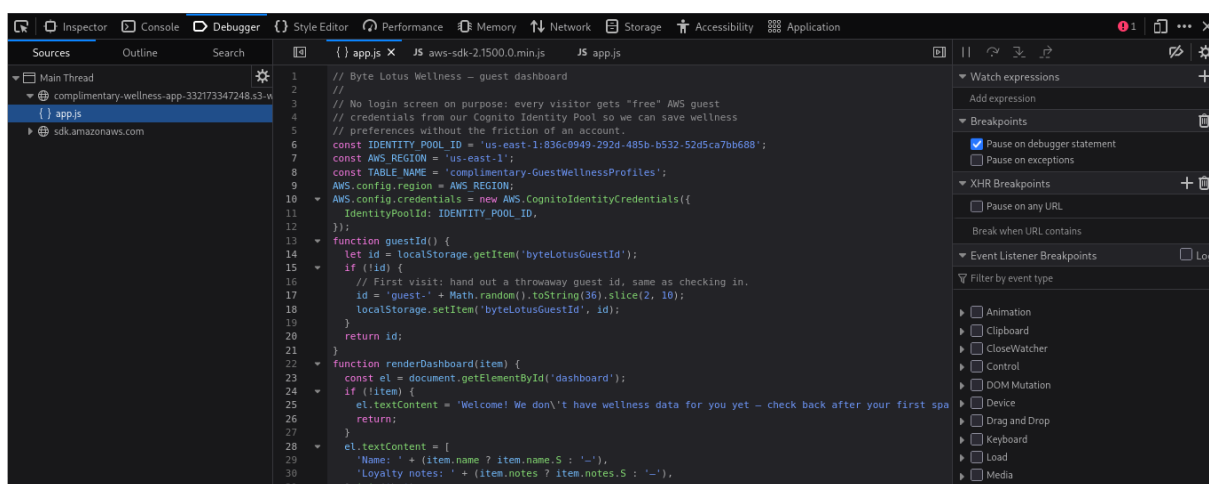


So one thing we can do now is inspect the page source (CTRL+U), but unfortunately, nothing is interesting there either. So now we try to inspect the page.

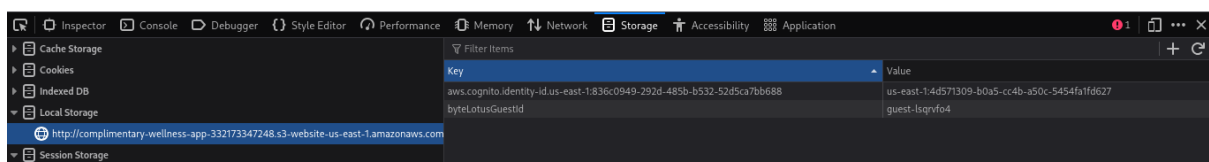


Here on the network tab (refresh the page) we see two interesting things: DynamoDB, which we can safely assume is a database or something related to a database by noticing the “db” at the end of the name. So we search that up and confirm that it is indeed an AWS database. Then we have Cognito-identity something-something. This surely must be related to how the app logs us in without requiring and registering or some sort of credentials.

But those are just bits of information, very useful ones but still bits. So we then look at the debugger tab and find app.js; this is very useful to understand how the app works. And there we inspect the code and find a pretty cool logic system.



Here we see that the app has the Identity pool ID on top, along with region and the table name. Then, as we look at the code, we see that the app creates a guest ID and then has some code for handling errors and stuff. Then, down below, there is some code that looks like it's for retrieving credentials from the DynamoDB database. Cookies can be found in the storage tab, so maybe we can find some useful session stuff there, and as suspected. Guest Id and Cognito ID are stored in the local storage.



With this much information, we can now start acting with some direction.

First thing I did (I would also recommend) was to type `aws help | grep "cognito"`. Since we'd typically not have much knowledge about this, we

```
urahara@Hogyoku:~$ aws help | grep "cognito"
```

After a bit of trying to use this option and some research, we get these credentials which we can use on the database. We already know that the database is DynamoDB from our earlier footprinting, and so again we type `aws help | grep "dynamodb"` and look at some information about these commands.

[illegible]

After exporting, we scan the given database table (The table name was given in the inspector page, debugger tab.) And then we got the passwords and all that stuff. And the flag.

```

$ aws dynamodb scan --table-name 'complimentary-GuestWellnessProfiles'
{
  "Items": [
    {
      "password": {
        "S": "digitaldetox2026"
      },
      "location": {
        "S": "25.2055,55.2733"
      },
      "notes": {
        "S": "Booked the quiet room for his \"digital detox.\" Checked email twice since writing that."
      }
    }
  ]
}

```

```
    "S": "escalation_only",
  },
  "location": {
    "S": "25.2048,55.2708"
  },
  "notes": {
    "S": "If you're reading this, the wellness app's guest role can read every profile, not just its own. THM{f[REDACTED]}"
  },
  "guest_id": {
    "S": "guest-vip-042"
  },
}
```

Notes & Lessons:

The app.js code contained the most crucial information for this assessment. And it was found not by sheer exploitation and brute force but through careful footprinting. For the most part, nothing active was done to get the information; the Cognito ID keys, region, and table name were found without any active footprinting. Information gathering was very useful.

The app.js exposed a lot of information that an attacker could use to compromise the database and steal sensitive data.